

Databases 51

SPARQL Evaluation, Query optimization

- Index Join
- In-memory Index join cost
- Join Order Optimization

Evaluation of query with single triple pattern

- Evaluation of single triple pattern is a primitive offered by a triplestore:
- ```
SELECT * {
 ?lecturer teaches ?course .
}
```

Result set:

- L1234567, C234567
- L1234567, C45678
- L7654321, C234567

```
L1234567 name Pat .
L7654321 name Vic .
L1234567 room 829 .
L7654321 room 567 .
C234567 programme GENED .
C45678 programme SOFTENG .
C234567 year 2019 .
C45678 year 2017 .
L1234567 teaches C234567 .
L1234567 teaches C45678 .
L7654321 teaches C234567 .
L7654321 assesses C45678 .
```

# Evaluation of query with single triple pattern

- Evaluation of single triple pattern is a primitive offered by a triplestore:
- The primitive will return the **triples**, i.e. a subgraph:
- ```
SELECT * {  
  ?lecturer teaches ?course .  
}
```

Result: a set of actual triples

- **L1234567 teaches C234567 .**
- **L1234567 teaches C45678 .**
- **L7654321 teaches C234567 .**

```
L1234567 name Pat .  
L7654321 name Vic .  
L1234567 room 829 .  
L7654321 room 567 .  
C234567 programme GENED .  
C45678 programme SOFTENG .  
C234567 year 2019 .  
C45678 year 2017 .  
L1234567 teaches C234567 .  
L1234567 teaches C45678 .  
L7654321 teaches C234567 .  
L7654321 assesses C45678 .
```

Query evaluation: single triple patterns

- Evaluation of single triple pattern is a primitive offered by a triplestore :
- `SELECT * WHERE {C45678 ?predicate ?object }`
- Possible 8 *abstract triple patterns*:

•	<code>?subject</code>	<code>?predicate</code>	<code>?object</code>	<code>XYZ</code>	
	<code>C45678</code>	<code>?predicate</code>	<code>?object</code>	<code>SYZ</code>	
	<code>?subject</code>	<code>?predicate</code>	<code>SOFTENG</code>	<code>XYO</code>	
	<code>?subject</code>	<code>programme</code>	<code>?object</code>	<code>XPZ</code>	
	<code>?subject</code>	<code>programme</code>	<code>SOFTENG</code>	<code>XPO</code>	} Most specific
	<code>C45678</code>	<code>programme</code>	<code>?object</code>	<code>SPZ</code>	
	<code>C45678</code>	<code>?predicate</code>	<code>SOFTENG</code>	<code>SYO</code>	
	<code>C45678</code>	<code>programme</code>	<code>SOFTENG</code>	<code>SPO</code>	

- Optimal query cost for single triple pattern: linear in size of result set
- Is achievable with enough index support! (but not always done)

Towards evaluating multi-pattern queries

- **SPARQL** query with two triple patterns and a shared variable is an equijoin and a self-join of the triple table:

- ```
SELECT * {
 ?lecturer name ?name .
 ?lecturer teaches ?course .
}
```

- Triple  $\bowtie$  equijoin on subject Triple

- How can this be achieved with the evaluation of single triple patterns?

**Triple(subject, predicate, object)**

```
L1234567 name Pat .
L7654321 name Vic .
L1234567 room 829 .
L7654321 room 567 .
C234567 programme GENED .
C45678 programme SOFTENG .
C234567 year 2019 .
C45678 year 2017 .
L1234567 teaches C234567 .
L1234567 teaches C45678 .
L7654321 teaches C234567 .
L7654321 assesses C45678 .
```

# Towards evaluating multi-pattern queries

- Natural approach: dealing with the triple patterns in the order that they appear in the query:
- `SELECT ?name {  
 ?lecturer name ?name . (a)  
 ?lecturer teaches ?course . (b)  
}`
- Triple  $\bowtie$  equijoin on subject Triple
- Cost of evaluating the first single triple pattern? XPZ 2
- Idea: **for each** result of (a),
- we have to find **all matching results** of (b)!

L1234567 name Pat .  
L7654321 name Vic .  
L1234567 room 829 .  
L7654321 room 567 .  
C234567 programme GENED .  
C45678 programme SOFTENG .  
C234567 year 2019 .  
C45678 year 2017 .  
L1234567 teaches C234567 .  
L1234567 teaches C45678 .  
L7654321 teaches C234567 .  
L7654321 assesses C45678 .

# Towards evaluating multi-pattern queries

- **Naive evaluation as Nested Loop.** Makes no use of current chosen match in outer loop.

```
• SELECT ?name {
 ?lecturer name ?name . (a)
 ?lecturer teaches ?course . (b)
}
```

```
for (each result of (a)) {
 for (each result of (b)) {
 check equijoin;
 process filter;
 project;
 output;
 }
}
```

```
L1234567 name Pat .
L7654321 name Vic .
L1234567 room 829 .
L7654321 room 567 .
C234567 programme GENED .
C45678 programme SOFTENG .
C234567 year 2019 .
C45678 year 2017 .
L1234567 teaches C234567 .
L1234567 teaches C45678 .
L7654321 teaches C234567 .
L7654321 assesses C45678 .
```

# Towards evaluating multi-pattern queries

- **Idea:** The effect of outer loops is a variable binding and a possible change of abstract triple patterns.

```
• SELECT * {
 ?lecturer name ?name . (a)
 ?lecturer teaches ?course . (b)
}
```

```
for (each result of (a)) {
 for (each matching result of (b)) {
 process filter; ...
 }
}
```

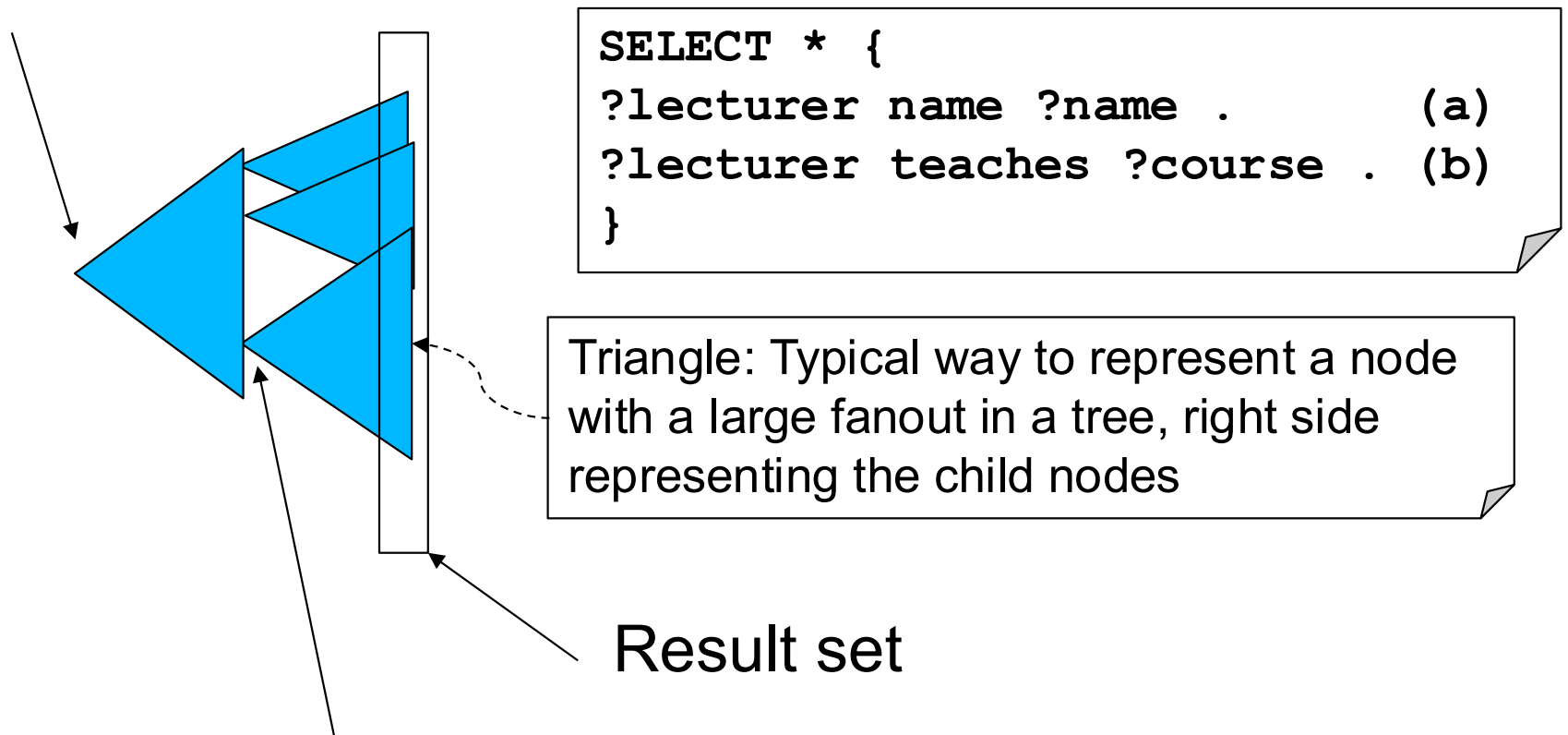
equijoin

- matching: Variable binding:
- In (b): **?lecturer is known** from (a):
- (b) is not XPZ, but SPZ

```
L1234567 name Pat .
L7654321 name Vic .
L1234567 room 829 .
L7654321 room 567 .
C234567 programme GENED .
C45678 programme SOFTENG .
C234567 year 2019 .
C45678 year 2017 .
L1234567 teaches C234567 .
L1234567 teaches C45678 .
L7654321 teaches C234567 .
L7654321 assesses C45678 .
```

# Query execution

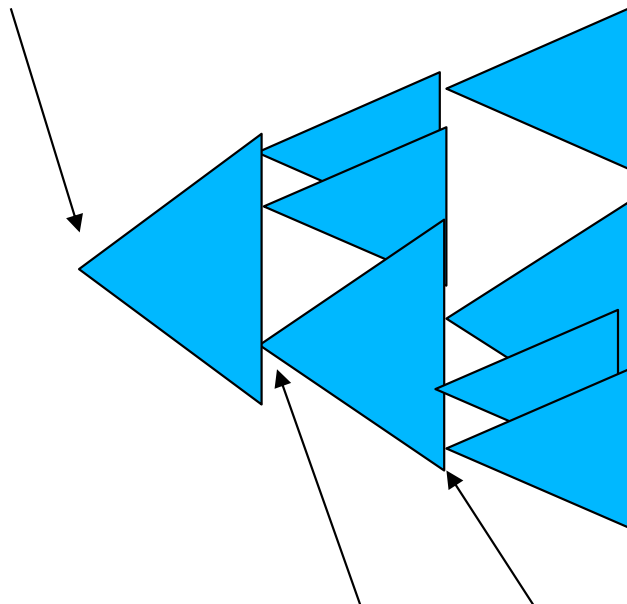
- Evaluation of Sparql Query as nested for loops:
- Outermost for loop of (a):



- for each step of one for loop: new inner for loop for (b)

# Query execution

- Evaluation of Sparql Query as nested for loops:
- Outermost for loop of (a):

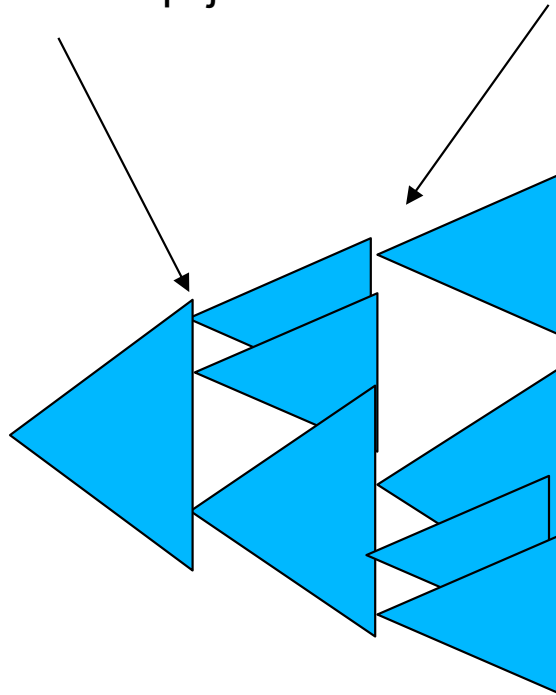


```
SELECT * {
 ?lecturer name ?name . (a)
 ?lecturer teaches ?course . (b)
 ?course programme ?p (c)
}
```

- for each step of one for loop: new inner for loop for (b)
- (c) ?course is known,
- so SPZ single triple pattern query

# As nested Join

- $(\text{Triple} \bowtie_{\text{equijoin}} \text{Triple}) \bowtie_{\text{equijoin}} \text{Triple}$



```
SELECT * {
 ?lecturer name ?name . (a)
 ?lecturer teaches ?course . (b)
 ?course programme ?p (c)
}
```

# Estimating Cost

- We can **estimate** the cost of a query plan if we have the average cost of the abstract triple patterns for a given predicate:
- E.g. For **teaches**:
- Each course is taught by an average of 2.2 lecturers.
- Each lecturer teaches an average of 4.2 courses.
- We denote this in the following form:  
**avg 2.2 (?lecturer teaches ?course) avg 4.2**
- If there is only one assessor then we have  
**1 (?lecturer assesses ?course) avg 1.9**

# Costs and Cost metrics notation we use

- Each course is taught by an average of 2.2 lecturers
- Each lecturer teaches an average of 4.2 courses.
- We denote this in the following form:

avg 2.2 (?lecturer teaches ?course) avg 4.2

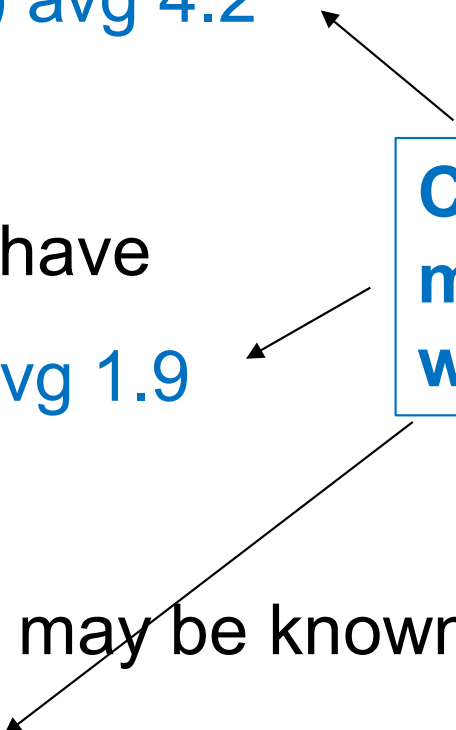
- If there is only one assessor then we have

1 (?lecturer assesses ?course) avg 1.9

- For constant subjects or objects: cost may be known:

2 (?lecturer teaches C234567)

**Cost  
metrics  
we use**



# Cost with larger basic graph pattern.

- avg 2.2 (?lecturer teaches ?course) avg 4.2
- 1 (?lecturer assesses ?course) avg 1.9
- 40 (?course programme SOFTENG)
- 60 (?course programme GENED)
- **SELECT \* {  
  ?course2 programme GENED  
  ?lecturer teaches ?course2 .  
  ?lecturer assesses ?course1 .  
  ?course1 programme SOFTENG .  
}**

# Cost with larger basic graph pattern.

- avg 2.2 (?lecturer teaches ?course) avg 4.2
- 1 (?lecturer assesses ?course) avg 1.9
- 40 (?course programme SOFTENG)
- 60 (?course programme GENED)

• **SELECT \* {**  
    ?course2 programme GENED                      60                      = 60  
    ?lecturer teaches ?course2 .                      \*2.2                      = 132  
    ?lecturer assesses ?course1 .                      \*1.9                      < 255  
    ?course1 programme SOFTENG .                      \*1                      < 255  
    }  
  
**Abstract Triple Pattern SPO**  
**Conservative est.** < 702

# Summary

- **Query optimization: Join order optimization**
- The order of the triple patterns in SPARQL give a simple linear query execution plan.
- Depending on the order of the triples in this plan the cost can differ.
- Previous choices in the plan bind variables and change the effective abstract triple pattern.
- With cost metrics we can estimate cost of query.